

Recurrent Neural Networks (RNN)

- **Recurrent Neural Networks (RNNs)** are a class of neural networks designed for **sequential data** (text, speech, time-series, video frames).
- Unlike **Feedforward Neural Networks (FNNs)** and **Convolutional Neural Networks (CNNs)**, RNNs have a **memory mechanism** that helps them remember **previous inputs**.
- They are widely used in **Natural Language Processing (NLP)**, **speech recognition**, **music generation**, and **time-series forecasting**.

Sequence Data

- Data that has **order** and **context matters**.
- Example: A sentence $\rightarrow$ “*I love AI*” $\rightarrow$ the word “love” depends on “I”.

Hidden State

- At each time step, the RNN maintains a **hidden state** (h_t).
- This hidden state captures **information from previous inputs**.
- Formula:

$$h_t = f(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

where:

- h_t = hidden state at time t
- x_t = input at time t
- W_h, W_x = weight matrices
- f = activation function (usually tanh/ReLU)

Timestep

- One **unit of time** in sequence processing.
- Example: For text input, each word is a timestep.

Input Feature

- The dimension of each input.
- Example: Word embeddings (vector representation of words) $\rightarrow$ each word is represented as a vector of numbers.

Output

- RNN can produce:
 - **Many-to-One**: Sentiment analysis (entire sentence $\rightarrow$ one label).

- **Many-to-Many**: Language translation (sequence → sequence).

Variants of RNN

- **Vanilla RNN** → basic RNN, suffers from vanishing gradient problem.
- **LSTM (Long Short-Term Memory)** → solves long-term dependency issue with gates (input, forget, output).
- **GRU (Gated Recurrent Unit)** → simpler than LSTM but effective.

Uses in Development

- **NLP**: Sentiment analysis, machine translation, text generation.
- **Speech Recognition**: Convert audio sequence to text.
- **Time-Series Forecasting**: Predict stock prices, weather data.
- **Music Generation**: Learn sequences of notes.

Simple Python Code Example

Here's a **basic RNN example with Keras** to demonstrate sequential input:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding

# Sample: Sequence classification
# Suppose we have 10 sequences, each with 5 timesteps, vocabulary
size = 20
X = np.random.randint(20, size=(10, 5)) # Input sequences (10
samples, 5 timesteps)
y = np.random.randint(2, size=(10, 1)) # Binary output (labels)

# Build RNN model
model = Sequential()
model.add(Embedding(input_dim=20, output_dim=8, input_length=5)) #
Word embedding
model.add(SimpleRNN(16, activation='tanh')) # RNN layer
model.add(Dense(1, activation='sigmoid')) # Output layer

# Compile model
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train
```

```
model.fit(X, y, epochs=5, verbose=1)
```

```
# Prediction
```

```
print("Sample Prediction:", model.predict(X[:1]))
```

Explanation of Code

1. Embedding Layer: Converts integer-encoded words $\rightarrow$ dense vectors.

2. SimpleRNN Layer: Processes sequences one timestep at a time, remembers past states. **3.**

Dense Layer: Outputs final classification.

4. Prediction: Outputs probability (0–1) for binary class.

Sequence Data :

- **Sequence Data** = Data where **order matters**.

- Each element depends on the **previous element(s)**.

- Examples:

- **Text:** A sentence $\rightarrow$ meaning depends on word order.

- *"I love AI" $\neq$ "AI love I"*

- **Audio:** A speech signal $\rightarrow$ each sound depends on previous sounds.

- **Time-series:** Stock price today depends on yesterday's price.

In **normal feedforward neural networks**, inputs are **independent**. But in **sequence tasks**, we need **context**.

How RNN Handles Sequence Data

Step-by-Step Process

1. Input sequence is fed **one element at a time** (word by word, timestep by timestep).

2. At each timestep **t**, RNN takes:

- Current input (x_t)

- Previous hidden state (h_{t-1}) (memory from past)

- Combines them to compute **new hidden state** (h_t)

3. Hidden state (h_t) = acts like **short-term memory**.

4. At the end, RNN can output:

- Final result (e.g., sentiment of whole sentence)

- Or sequence of outputs (e.g., translation word-by-word).

Memory Mechanism in RNN

- **Hidden State** = Memory cell that carries **past information**.

- Formula:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

-
-

where:

- x_t = current input
- h_{t-1} = previous memory
- h_t = new memory
- f = activation function (tanh, ReLU)

Example

Sentence: “**I love AI**”

- At timestep 1: input = “I”, memory starts forming.
- At timestep 2: input = “love”, RNN remembers “I” + “love”.
- At timestep 3: input = “AI”, RNN remembers “I love AI”.

Thus, RNN builds meaning step-by-step.

Simple Visualization

```
x1 → [RNN Cell] → h1  
x2 → [RNN Cell] → h2  
x3 → [RNN Cell] → h3 → output
```

- Each **RNN cell** passes **memory (h)** forward.
- Like a **chain** of connected cells.

Small Python Example to Demonstrate Memory

Here’s a **toy RNN implementation** to show how it processes a sequence:

```
import numpy as np
```

```
# Example sentence: "I love AI" (encoded as  
numbers) sequence = [1, 2, 3] # Let's say 1=I,
```

2=love, 3=AI

```
# Initialize weights and hidden state
Wx, Wh, b = 0.5, 0.8, 0.1 # random chosen values
h = 0 # initial hidden state

print("Processing sequence step by step:")

for t, x in enumerate(sequence):
    h = np.tanh(Wx * x + Wh * h + b) # memory update
    print(f"Timestep {t+1} | Input={x} | Hidden State={h:.4f}")
```

Output Explanation

- At **timestep 1**, RNN only sees “I” → stores memory.
- At **timestep 2**, it sees “love” + remembers “I”.
- At **timestep 3**, it sees “AI” + remembers “I love”.

This demonstrates **sequence + memory flow**.

Why This Matters in Development

- RNNs are powerful because they **don’t forget the past immediately**.
- Example applications:
 - **Chatbot** → remembers previous words in conversation.
 - **Stock predictor** → remembers yesterday’s price.
 - **Music generator** → remembers last few notes.

Advantages & Limitations

Advantages:

- Good for sequential data.
- Captures context (history).

Limitations:

- **Vanishing Gradient** → hard to remember long-term dependencies.
- Slow for very long sequences.
- Replaced in modern NLP by **Transformers (e.g., BERT, GPT, ChatGPT)**.

Summary

- RNNs are neural networks for **sequential tasks**.
- They maintain **memory** using hidden states.
- Widely used in **NLP, speech, and forecasting**.
- Advanced versions: **LSTM & GRU**.
- Modern replacements: **Transformers**.